

Resumen integral y unificado de diseño de pruebas

Básiacmente se dividen en dos grupos principales, verificación (si el código hace lo que debe) y validación (si el código es lo que necesita el usuario).

Pruebas Estáticas y Manuales

- **Estáticas**, que se integran con el uso de linters para la detección de errores de estilo. También pueden estar en la forma de revisiones de código, validando la arquitectura y la lógica previo a integrar cualquier cosa.
- **Manuales**, como la validación directa en navegadores o herramientas como Postman. En conjunto con las **pruebas de usabilidad** (como la técnica de pensar en voz alta) son esenciales para descubrir fallas de diseño que las pruebas automatizadas no detectan.

Pruebas de Caja Blanca y Unitarias

- **Diseño**, identificando **clases de equivalencia** (que el sistema trata igual) y **valores frontera** (probar los límites de una condición).
- **Implementación**, Se automatizan con frameworks como **Jest**, (utilizando el **patrón AAA: Arrange, Act, Assert**) para estructurar cada caso de prueba de forma clara y aislada.

Pruebas de Integración

Verifican correcta integración ente módulos, y, a diferencia de las unitarias, suelen usar dependencias reales.

Integración en el Proceso de Desarrollo

El flujo ideal comienza con el diseño de casos de prueba incluso antes de programar, basándose en los casos de uso. Posteriormente, se integran sistemas de **CI/CD** (como GitHub Actions) que ejecutan automáticamente sus tests en cada PR antes de realizar el merge a la rama principal y que permiten seguir desarrollando a la vez que se está integrando y viceversa.

Revisiones de código

Tras la primera entrega del sistema para el socio formador, hubo correcciones requeridas en las siguientes funcionalidades:

- Error al crear SOFOM
- Error en la usabilidad al registrar a un cliente (si un dato era erróneo, se borraba lo ingresado)
- Oficial de cumplimiento no podía registrar cliente

Dichas funcionalidades fueron corregidas para la siguiente iteración.

Otros de los cambios que hubo fueron los de algunos modelos, pues sus queries desoptimizadas con joins a la tabla de catálogos (con más de 9k registros), provocaron problemas con el heap en node.

También hubo que crear algunos índices para mejorar tiempos de respuesta de la BD.

Se llegó a la decisión de todo cambio tras alguna de las siguientes situaciones:

- Socio formador presentó inconformidad
- Diagnóstico de queries y rendimiento en supabase
- Problemas con el heap en node

Aplicación de la metodología para elaborar casos de prueba

ID	Descripción	Precondición	Entrada	Acción	Salida esperada	Aspecto de usabilidad favorecido
CP-LOGIN-01	Inicio de sesión con credenciales válidas	Usuario registrado y activo	Correo y contraseña correctos	Enviar formulario de login	Acceso al dashboard según rol y permisos ABAC	Acceso rápido y sin ambigüedades
CP-LOGIN-02	Rechazo de credenciales inválidas	Usuario registrado	Contraseña incorrecta	Intentar iniciar sesión	Mensaje de error y permanencia en login	Prevención de errores y retroalimentación clara
CP-FNC00-01	Registro exitoso de nueva SOFOM	Administrador autenticado	Datos válidos de SOFOM	Registrar SOFOM	SOFOM almacenada en BD y visible en catálogo	Facilidad de administración
CP-FNC00-02	Registro de usuario con rol asignado	Administrador autenticado y SOFOM existente	Datos de usuario y rol	Crear usuario	Usuario registrado con permisos ABAC correspondientes	Gestión consistente de accesos
CP-FNC00-03	Validación de campos obligatorios en alta de usuario	Administrador autenticado	Correo vacío o inválido	Intentar guardar usuario	Error de validación y no se almacena información	Prevención de captura incorrecta
CP-FNC01-01	Alta manual de cliente	Usuario con acceso a módulo A2	Datos completos de cliente	Guardar cliente	Cliente almacenado correctamente	Flujo de captura eficiente
CP-FNC01-03	Consulta de expediente de cliente	Cliente existente	ID de cliente	Abrir expediente	Visualización de datos, contratos, operaciones y alertas asociadas	Acceso centralizado a información

CP-FNC05-01	Generación automática de alerta por coincidencia en lista bloqueada	Cliente y listas de riesgo cargadas	Cliente presente en lista	Ejecutar job de revisión	Creación de alerta ROI-24 y notificación al OC	Seguridad y monitoreo oportuno
CP-FNC09-02	Operación fuera de criterios	Regla activa	Operación normal	Ejecutar evaluación	No se genera alerta	Disminución de ruido operativo
CP-FNC12-03	Validación de parámetros inválidos	OC autenticado	Umbral negativo o vacío	Guardar regla	Mensaje de error y rechazo	Prevención de configuraciones erróneas
CP-FNC13-01	Generación de reporte ROR	Existe operación relevante	Datos de operación	Generar reporte	Archivo ROR generado con formato correcto	Automatización documental
CP-FNC13-02	Generación de reporte ROI derivado de alerta aprobada	Alerta aprobada por OC	ID de alerta	Emitir reporte	Reporte ROI almacenado y disponible	Trazabilidad regulatoria
CP-ABAC-01	Restricción de acceso por rol	Usuario con permisos limitados	Intento de acceso a módulo restringido	Navegar a ruta protegida	Acceso denegado	Seguridad y claridad de permisos
CP-ROIP-01	Creación de reporte interno con evidencia	Usuario autorizado	Asunto, descripción y PDF	Enviar reporte interno	Reporte almacenado y evidencia asociada	Facilidad de denuncia y cumplimiento

Inicio de sesión

Clases de equivalencia				Valores frontera			Casos de prueba					
Clase	Entrada	Comportamiento esperado	Caso	Valor	¿Por qué este valor?	Resultado esperado	ID	Descripción	Precondición	Entrada	Acción	Salida esperada
Inválida	Correo vacío	No permite enviar formulario	""	correo = ""	Justo en el límite inferior del correo	No permite envío	CP-Login-01	Inicio de sesión exitoso	Usuario registrado	correo válido y contraseña válida	Presionar "Ingresar"	Redirección a dashboard
Inválida	Correo con formato incorrecto	No permite enviar formulario	usuario.com	correo = a@b	Justo debajo de un correo válido	Error de validación	CP-Login-02	Login con correo inexistente	Usuario no registrado	correo inexistente	Presionar "Ingresar"	Mostrar credenciales incorrectas
Inválida	Correo no registrado	Mostrar credenciales incorrectas	inexistente@correo.com	correo = a@b.c	Formato mínimo válido	Permite autenticación	CP-Login-03	Login con contraseña incorrecta	Usuario registrado	contraseña incorrecta	Presionar "Ingresar"	Mostrar credenciales incorrectas
Inválida	Contraseña incorrecta	Mostrar credenciales incorrectas	password Incorrecto	contraseña = 8 caracteres	Longitud mínima posible	Sistema procesa autenticación	CP-Login-04	Login con correo vacío	Ninguna	correo vacío	Presionar "Ingresar"	Navegador impide envío
Inválida	Contraseña incorrecta	Mostrar credenciales incorrectas	password Incorrecto	contraseña = 8 caracteres	Longitud mínima posible	Sistema procesa autenticación	CP-Login-05	Login con contraseña vacía	Ninguna	contraseña vacía	Presionar "Ingresar"	Navegador impide envío
Válida	Credenciales correctas	Inicio de sesión exitoso	usuario@correo.com / password Correcto	contraseña = 255 caracteres	Contraseña extensa	Sistema responde sin fallar	CP-Login-06	Login con correo inválido	Ninguna	usuario.com	Presionar "Ingresar"	Error de validación
							CP-Login-07	Verificar creación de sesión	Usuario registrado	credenciales válidas	Iniciar sesión	Sesión creada correctamente

Logout

Clases de equivalencia				Valores frontera			Casos de prueba					
Clase	Entrada	Comportamiento esperado	Caso	Valor	¿Por qué este valor?	Resultado esperado	ID	Descripción	Precondición	Entrada	Acción	Salida esperada
Válida	Usuario autenticado	Cierra sesión correctamente	POST /logout	Sesión activa	Límite válido	Sesión destruida	CP-Logout-01	Logout exitoso	Usuario autenticado	POST /logout	Ejecutar logout	Redirección a login
Inválida	Usuario sin sesión	Redirección sin error	Logout sin sesión	null	Ausencia total de sesión	No falla	CP-Logout-02	Verificar destrucción de sesión	Usuario autenticado	Logout	Ejecutar logout	Sesión eliminada
Inválida	Método GET	No ejecuta logout	GET /logout	POST	Método permitido	Logout exitoso	CP-Logout-03	Acceso después de logout	Usuario cerró sesión	Acceder a dashboard	Navegar al sistema	Redirección a login
				GET	Justo fuera del método válido	Logout no ejecutado	CP-Logout-04	Logout sin sesión activa	Usuario no autenticado	POST /logout	Ejecutar logout	No ocurre error
							CP-Logout-05	Logout mediante GET	Ninguna	GET /logout	Acceder a ruta	Logout no permitido

Registrar usuario

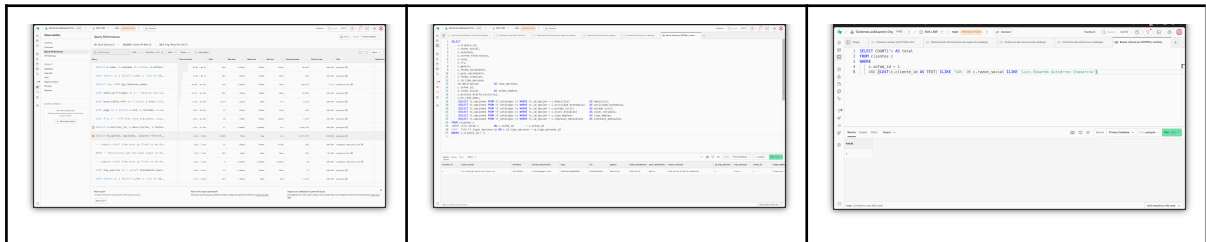
Clases de equivalencia	Valores frontera	Casos de prueba
------------------------	------------------	-----------------

Clase	Entrada	Comportamiento esperado	Caso	Valor	¿Por qué este valor?	Resultado esperado	ID	Descripción	Precondición	Entrada	Acción	Salida esperada
Inválida	Nombre vacío	No permite enviar formulario	""	""	Nombre vacío	No permite envío	CP-RegUsuario-01	Registro exitoso de usuario	Correo no registrado	nombre válido, correo válido y contraseña válida	Presionar "Crear usuario"	Usuario registrado correctamente
Inválida	Correo vacío	No permite enviar formulario	""	1 carácter	Longitud mínima del nombre	Sistema procesa registro	CP-RegUsuario-02	Registro con nombre vacío	Ninguna	nombre vacío	Presionar "Crear usuario"	Navegador impide envío
Inválida	Correo con formato incorrecto	No permite enviar formulario	usuario.com	""	Correo vacío	No permite envío	CP-RegUsuario-03	Registro con correo vacío	Ninguna	correo vacío	Presionar "Crear usuario"	Navegador impide envío
Inválida	Contraseña vacía	No permite enviar formulario	""	a@b	Justo debajo de un correo válido	Error de validación	CP-RegUsuario-04	Registro con correo inválido	Ninguna	usuario.com	Presionar "Crear usuario"	Error de validación
Inválida	Correo ya registrado	Mostrar mensaje de usuario existente	usuario@correo.com repetido	a@b.c	Formato mínimo válido	Permite registro	CP-RegUsuario-05	Registro con contraseña vacía	Ninguna	contraseña vacía	Presionar "Crear usuario"	Navegador impide envío
Válida	Datos completos y correctos	Registro exitoso	usuario@correo.com / password123	8 caracteres	Contraseña vacía	No permite envío	CP-RegUsuario-06	Registro con correo ya existente	Usuario previamente registrado	correo repetido	Presionar "Crear usuario"	Mostrar mensaje de usuario existente
Válida	Contraseña válida	Contraseña almacenada cifrada	password123	255 caracteres	Entrada extensa	Sistema responde sin fallar	CP-RegUsuario-07	Registro con contraseña de 8 caracteres	Ninguna	contraseña de 8 caracteres	Presionar "Crear usuario"	Sistema procesa registro
							CP-RegUsuario-08	Registro con campos extensos	Ninguna	entradas de 255 caracteres	Presionar "Crear usuario"	Sistema responde sin fallar

Bitácora de ejecución de pruebas

Derivado de las correcciones de código, se ejecutaron pruebas manuales y de integración previo a la corrección de errores, para verificar la falla y saber qué módulos estaban involucrados en esta.

- Se diseñaron los casos de prueba
- Se ejecutaron las pruebas
 - La mayoría de las pruebas fueron manuales (directamente realizadas en el navegador)
 - Pruebas realizadas sobre queries de modelos, en supabase (así como un monitoreo del rendimiento de determinados queries que supabase cataloga como lentas).



- Otras pruebas fueron unitarias, para algunas funcionalidades:
 - Login

ID	Descripción	Técnica	Precondición	Entrada	Acción	Salida esperada
CP-Login-01	Inicio de sesión exitoso (rol no Root)	Clase válida	Usuario registrado en BD	correo_electronico: 'ana@valaw.com' / contraseña: 'Password1!'	postLogin()	res.redirect('/alertas')
CP-Login-01b	Inicio de sesión exitoso (rol Root)	Clase válida	Usuario Root registrado en BD	correo_electronico: 'root@valaw.com' / contraseña: 'RootPass1!'	postLogin()	res.redirect('/sofomes')
CP-Login-02	Correo no registrado	Clase inválida	Usuario no existe en BD	correo_electronico: 'inexistente@correo.com' / contraseña cualquiera	postLogin()	render('auth/login', { error: 'Credenciales incorrectas' })
CP-Login-03	Contraseña incorrecta	Clase inválida	Usuario registrado en BD	correo_electronico válido / contraseña: 'wrongpassword'	postLogin()	render('auth/login', { error: 'Credenciales incorrectas' })
CP-Login-04	Correo vacío — frontera inferior	Valor frontera	Ninguna	correo_electronico: '' / contraseña cualquiera	postLogin()	render('auth/login', { error: 'Credenciales incorrectas' })
CP-Login-05	Contraseña vacía — frontera inferior	Valor frontera	Usuario registrado en BD	correo_electronico válido / contraseña: ''	postLogin()	render('auth/login', { error: 'Credenciales incorrectas' })
CP-Login-06	Correo sin formato @	Clase inválida	Ninguna	correo_electronico: 'usuario.com' / contraseña cualquiera	postLogin()	render('auth/login', { error: 'Credenciales incorrectas' })
CP-Login-07	Sesión creada correctamente	Postcondición	Usuario registrado en BD	Credenciales válidas	postLogin()	req.session.usuario contiene { usuario_id, nombre, correo, rol, sofom_id }
CP-Login-08	Contraseña de 8 caracteres — frontera mínima	Valor frontera	Usuario registrado en BD	contraseña: '12345678' (8 chars)	postLogin()	bcrypt.compare es invocado, res.redirect('/alertas')

CP-Login-09	Contraseña de 255 caracteres — frontera superior	Valor frontera	Usuario registrado en BD	contrasena: 'A'.repeat(255)	postLogin()	Sistema responde sin error, res.redirect('/alertas')
-------------	---	----------------	--------------------------	-----------------------------	-------------	---

JavaScript

```

jest.mock('../util/database');
jest.mock('../models/usuario.model');
jest.mock('bcryptjs');

const bcrypt = require('bcryptjs');
const Usuario = require('../models/usuario.model');

const authController = require('../controllers/auth.controller');

function buildReq(body = {}) {
  return {
    body,
    session: {},
  };
}

function buildRes() {
  const res = {
    _rendered: null,
    _redirected: null,
    _status: 200,
    status(code) { this._status = code; return this; },
    render(view, locals) { this._rendered = { view, locals }; },
    redirect(url) { this._redirected = url; },
    send(body) { this._sent = body; },
  };
  return res;
}

const USUARIO_DB = {
  usuario_id: 7,
  nombre: 'Ana López',
  correo_electronico: 'ana@svalaw.com',
  contrasena: '$2b$10$hashedpassword',
  rol_id: 2,
  rol: 'Empleado',
  sofof_id: 1,
};

const USUARIO_ROOT_DB = {
  ...USUARIO_DB,
  usuario_id: 1,
  rol_id: 1,
  rol: 'Root',
  sofof_id: null,
};

function mockUsuarioEncontrado(usuario = USUARIO_DB) {
  Usuario.fetchByEmail.mockResolvedValue({ rows: [usuario] });
}

function mockUsuarioNoEncontrado() {
  Usuario.fetchByEmail.mockResolvedValue({ rows: [] });
}

function mockContrasenaCorrecta() {
  bcrypt.compare.mockResolvedValue(true);
}

function mockContrasenaIncorrecta() {
  bcrypt.compare.mockResolvedValue(false);
}

beforeEach(() => {
  jest.clearAllMocks();
});

describe('getLogin renderiza el formulario', () => {
  test('Renderiza auth/login con error = null', () => {
    const req = buildReq();
    const res = buildRes();

    authController.getLogin(req, res);

    expect(res._rendered.view).toBe('auth/login');
    expect(res._rendered.locals).toEqual({ error: null });
  });
});

describe('postLogin - autenticación de usuarios', () => {
  test('CP-Login-01 Clase válida: credenciales correctas redirect /alertas', async () => {
    mockUsuarioEncontrado();
    mockContrasenaCorrecta();

    const req = buildReq({
      correo_electronico: 'ana@svalaw.com',
      contrasena: 'Password!!',
    });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(Usuario.fetchByEmail).toHaveBeenCalledWith('ana@svalaw.com');

    expect(bcrypt.compare).toHaveBeenCalledWith('Password!!', USUARIO_DB.contrasena);

    expect(res._redirected).toBe('/alertas');
    expect(res._rendered).toBeNull();
  });

  test('CP-Login-01b | Usuario Root - redirect /sofof', async () => {
    mockUsuarioEncontrado(USUARIO_ROOT_DB);
    mockContrasenaCorrecta();

    const req = buildReq({
      correo_electronico: 'root@svalaw.com',
      contrasena: 'RootPass!!',
    });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(res._redirected).toBe('/sofof');
  });

  test('CP-Login-02 Clase inválida: correo no registrado "Credenciales incorrectas"', async () => {
    mockUsuarioNoEncontrado();

    const req = buildReq({
      correo_electronico: 'inexistente@correo.com',
      contrasena: 'cualquierCosa',
    });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(bcrypt.compare).not.toHaveBeenCalled();

    expect(res._rendered.view).toBe('auth/login');
    expect(res._rendered.locals.error).toBe('Credenciales incorrectas');
    expect(res._redirected).toBeNull();
  });

  test('CP-Login-03 Clase inválida: contraseña incorrecta "Credenciales incorrectas"', async () => {
    mockUsuarioEncontrado();
    mockContrasenaIncorrecta();

    const req = buildReq({

```

```

        correo_electronico: 'ana@svlaw.com',
        contrasena: 'wrongpassword',
    });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(bcrypt.compare).toHaveBeenCalledTimes(1);

    expect(res._rendered.view).toBe('auth/login');
    expect(res._rendered.locals.error).toBe('Credenciales incorrectas');
});

test('CP-Login-04 Frontera inferior: correo = "" "Credenciales incorrectas", async () => {
    mockUsuarioNoEncontrado();

    const req = buildReq({ correo_electronico: '', contrasena: 'algo' });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(Usuario.fetchByEmail).toHaveBeenCalledTimes(1);
    expect(res._rendered.locals.error).toBe('Credenciales incorrectas');
});

test('CP-Login-05 Frontera inferior: contrasena = "" "Credenciales incorrectas", async () => {
    mockUsuarioEncontrado();
    mockContrasenaIncorrecta();

    const req = buildReq({ correo_electronico: 'ana@svlaw.com', contrasena: '' });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(bcrypt.compare).toHaveBeenCalledTimes(1);
    expect(res._rendered.locals.error).toBe('Credenciales incorrectas');
});

test('CP-Login-06 Clase inválida: correo "usuario.com" sin @ "Credenciales incorrectas", async () => {
    mockUsuarioNoEncontrado();

    const req = buildReq({ correo_electronico: 'usuario.com', contrasena: 'algoCualquiera' });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(res._rendered.view).toBe('auth/login');
    expect(res._rendered.locals.error).toBe('Credenciales incorrectas');
});

test('CP-Login-07 Credenciales válidas req.session.usuario contiene los campos correctos', async () => {
    mockUsuarioEncontrado();
    mockContrasenaCorrecta();

    const req = buildReq({
        correo_electronico: 'ana@svlaw.com',
        contrasena: 'Password1!',
    });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(req.session.usuario).toEqual({
        usuario_id: USUARIO_DB.usuario_id,
        nombre: USUARIO_DB.nombre,
        correo: USUARIO_DB.correo_electronico,
        rol: USUARIO_DB.rol,
        sofom_id: USUARIO_DB.sofom_id,
    });
});

test('CP-Login-08 Frontera: contraseña de exactamente 8 caracteres sistema procesa autenticación', async () => {
    mockUsuarioEncontrado();
    mockContrasenaCorrecta();

    const req = buildReq({
        correo_electronico: 'ana@svlaw.com',
        contrasena: '12345678',
    });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(bcrypt.compare).toHaveBeenCalledTimes(1);
    expect(res._redirected).toBe('/alertas');
});

test('CP-Login-09 Frontera superior: contraseña de 255 caracteres sistema responde sin fallar', async () => {
    mockUsuarioEncontrado();
    mockContrasenaCorrecta();

    const pass255 = 'A'.repeat(255);
    const req = buildReq({
        correo_electronico: 'ana@svlaw.com',
        contrasena: pass255,
    });
    const res = buildRes();

    await authController.postLogin(req, res);

    expect(bcrypt.compare).toHaveBeenCalledTimes(1);
    expect(res._redirected).toBe('/alertas');
});
});

```

ii. Logout

ID	Descripción	Técnica	Precondición	Entrada	Acción	Salida esperada
CP-Logout-01	Cierre de sesión exitoso	Clase válida	Sesión activa (req.session.usuario poblado)	session.destroy ejecuta callback sin error	logout()	res.redirect('/login')
CP-Logout-02	Error al destruir la sesión	Clase inválida	Sesión activa, fallo interno del store (pg)	session.destroy ejecuta callback con Error('DB fail')	logout()	res.status(500).send('Error al cerrar sesión')
CP-Logout-03	Sesión ya destruida / vacía	Clase inválida	req.session existe pero destroy lanza error inmediato	session.destroy callback con error	logout()	res.status(500).send('Error al cerrar sesión')
CP-Logout-04	destroy es invocado exactamente una vez	Postcondición	Sesión activa	Cualquier estado del callback	logout()	session.destroy llamado exactamente 1 vez
CP-Logout-05	No se hace redirect si destroy falla	Clase inválida	Sesión activa, fallo en destroy	session.destroy con error	logout()	res.redirect no es llamado
CP-Logout-06	No se envía respuesta 500 si destroy es exitoso	Clase válida	Sesión activa	session.destroy sin error	logout()	res.status(500) no es llamado

JavaScript

```
jest.mock('../util/database');
jest.mock('../models/usuario.model');
jest.mock('bcryptjs');

const authController = require('../controllers/auth.controller');

function buildRes() {
  const res = {
    _status: null,
    _sent: null,
    _redirected: null,
    status(code) {
      this._status = code;
      return this;
    },
    send(body) {
      this._sent = body;
      return this;
    },
    redirect(url) {
      this._redirected = url;
    },
  };
  return res;
}

function buildReq({ destroyError = null } = {}) {
  return {
    session: {
      usuario: { usuario_id: 7, nombre: 'Ana', rol: 'Empleado', sofom_id: 1 },
      destroy: jest.fn(cb => cb(destroyError)),
    },
  };
}

beforeEach(() => {
  jest.clearAllMocks();
});

describe('Logout cierre de sesión', () => {
  test('CP-Logout-01 Clase válida: destroy sin error redirect /login', () => {
    const req = buildReq({ destroyError: null });
    const res = buildRes();

    authController.logout(req, res);

    expect(res._redirected).toBe('/login');
  });

  test('CP-Logout-02 Clase inválida: destroy con Error status 500 + mensaje', () => {
    const req = buildReq({ destroyError: new Error('DB connection lost') });
    const res = buildRes();

    authController.logout(req, res);

    expect(res._status).toBe(500);
    expect(res._sent).toBe('Error al cerrar sesión');
  });

  test('CP-Logout-03 Clase inválida: Error("") sigue siendo truthy status 500', () => {
    const req = buildReq({ destroyError: new Error('') });
    const res = buildRes();

    authController.logout(req, res);

    expect(res._status).toBe(500);
    expect(res._sent).toBe('Error al cerrar sesión');
  });

  test('CP-Logout-04 Postcondición: session.destroy es invocado exactamente 1 vez', () => {
    const req = buildReq({ destroyError: null });
    const res = buildRes();

    authController.logout(req, res);

    expect(req.session.destroy).toHaveBeenCalledTimes(1);
  });

  test('CP-Logout-05 Clase inválida: destroy con error redirect NO es llamado', () => {
    const req = buildReq({ destroyError: new Error('fallo') });
    const res = buildRes();

    authController.logout(req, res);

    expect(res._redirected).toBeNull();
  });

  test('CP-Logout-06 Clase válida: destroy exitoso status 500 NO es llamado', () => {
    const req = buildReq({ destroyError: null });
    const res = buildRes();

    authController.logout(req, res);

    expect(res._status).toBeNull();
    expect(res._sent).toBeNull();
  });
});
```

Test coverage

All files

71.73% Statements 33/46 100% Branches 8/8 40% Functions 4/10 71.73% Lines 33/46

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
config	100%	1/1	100%	1/1
controllers	92.85%	26/28	100%	8/8
models	20%	2/10	0/0	0/4
util	57.14%	4/7	0/0	0/2

Evaluación heurística

Heurística de Nielsen	Observaciones
1. Visibilidad del estado del sistema	El sistema contempla alertas, notificaciones, estados de reportes, días restantes para dictamen, historial de alertas y registros de auditoría, lo que proporciona retroalimentación continua al usuario.
2. Correspondencia entre el sistema y el mundo real	La terminología utilizada corresponde al dominio regulatorio,, alineándose con el lenguaje de los usuarios expertos del sector financiero.
3. Control y libertad del usuario	Se contempla edición de información de clientes, parametrización de reglas y emisión de dictámenes mediante acciones explícitas de aprobar o rechazar.
4. Consistencia y estándares	Existe una guía formal de estilo para interfaz, código y base de datos; además, las vistas siguen una organización consistente basada en módulos funcionales.
5. Prevención de errores	El uso de listas desplegables para catálogos PLD/FT anula errores de captura en determinados campos; también se restringe la carga de documentos PDF y se aplican permisos ABAC.
6. Reconocimiento antes que recuerdo	La interfaz propuesta utiliza listados de clientes, tablas de alertas, menús desplegables y catálogos seleccionables, reduciendo la necesidad de memorizar valores.
7. Flexibilidad y eficiencia de uso	Se incluyen búsquedas, filtros, agrupamientos, automatización mediante jobs y parametrización de puntaje.
8. Diseño estético y minimalista	La arquitectura modular separa funciones por responsabilidades y roles, evitando exponer funcionalidades innecesarias según el nivel de acceso.
9. Ayudar a reconocer,	Contempla registros de auditoría y trazabilidad que facilitan el diagnóstico de incidentes.

diagnosticar y recuperarse de errores	
10. Ayuda y documentación	El sistema cuenta con glosario, casos de uso, reglas de negocio y documentación extensa del dominio, y contempla un manual.